Dưới đây là chi tiết từng bước thực hiện **Option A (OpenCV + Mô hình Camera Pinhole)** để phát hiện khối trụ tròn trên nền bảng trắng và xuất tọa độ 3D ((X, Y, Z)) so với camera.

BƯỚC 1: Hiệu chỉnh Camera (Camera Calibration)

- **Mục đích:** Tìm các thông số quang học nội tại (Intrinsics Matrix (K)) của camera.
- **Giải thích:** Camera chiếu khung cảnh 3D thực tế lên mặt phẳng ảnh 2D. Để quy đổi ngược từ pixel sang mét/cm, bạn cần biết ma trận Ma trận Đặc tính trong (K): [K = \begin{bmatrix} f_x & 0 & c_x \ 0 & f_y & c_y \ 0 & 0 & 1 \end{bmatrix}]
  - (f_x, f_y): Tiêu cự camera tính theo đơn vị pixel.
  - (c_x, c_y): Tọa độ tâm quang học (thường nằm gần giữa khung hình 2D).
- **Cách thực hiện:**
  1. In một tấm bảng bàn cờ caro (Chessboard).
  2. Chụp từ 10–20 bức ảnh tấm bàn cờ ở các góc độ khác nhau.
  3. Chạy hàm `cv2.calibrateCamera()` trong OpenCV để thu về ma trận (K) ((f_x, f_y, c_x, c_y)).

BƯỚC 2: Phát hiện khối trụ & Xác định tọa độ 2D (Image Processing & Centroid Extraction)

- **Mục đích:** Tách khối trụ khỏi tấm bảng trắng và xác định tâm pixel ((u, v)) cùng kích thước ảnh (w_{px}).
- **Giải thích:** Tấm bảng trắng tạo độ tương phản màu sắc/độ sáng rất cao so với khối trụ.
- **Cách thực hiện:**
  1. **Chuyển không gian màu:** Chuyển ảnh từ RGB sang không gian màu **HSV** hoặc ảnh xám (Grayscale).
  2. **Tách ngưỡng (Thresholding):** Dùng `cv2.inRange()` (hoặc `cv2.threshold()`) để biến bảng trắng thành màu đen và khối trụ thành màu trắng.
  3. **Tìm đường viền (Contours):** Dùng `cv2.findContours()` lấy đường viền vật thể. Lọc contour có diện tích lớn nhất (chính là khối trụ).
  4. **Xác định tâm ((u, v)) và chiều rộng (w_{px}):**
     - Lấy khung bao bằng `cv2.boundingRect(contour)` thu được `(x, y, w, h)`.
     - Tọa độ tâm pixel: (u = x + \frac{w}{2}), (v = y + \frac{h}{2}).
     - Kích thước chiều rộng khối trụ trên ảnh (pixel): (w_{px} = w).

BƯỚC 3: Tính khoảng cách độ sâu (Z) (Depth Calculation)

- **Mục đích:** Xác định khoảng cách từ camera đến khối trụ theo trục (Z).
- **Giải thích:** Dựa trên nguyên lý tam giác đồng dạng của mô hình camera Pinhole: [\frac{w*{px}}{f_x} = *frac{D*{real}}{Z}]
  - (D_{real}): Đường kính thực tế của khối trụ (đơn vị: mét hoặc cm - biết trước).
  - (w_{px}): Đường kính (chiều rộng) khối trụ đo được trên mặt phẳng pixel ảnh.
  - (f_x): Tiêu cự theo trục X thu được ở Bước 1.
- **Công thức tính (Z):** [Z = \frac{f*x* *times D*{real}}{w_{px}}]

BƯỚC 4: Chiếu ngược 2D sang 3D (Back-projection to (X, Y, Z))

- **Mục đích:** Tính tọa độ trục ngang (X) và trục đứng (Y) trong không gian 3D thực tế so với camera.
- **Giải thích:** Từ tọa độ điểm tâm trên ảnh ((u, v)) và khoảng cách độ sâu (Z), ta chiếu ngược tia sáng qua tâm quang học ((c_x, c_y)): [X = \frac{(u - c_x) \times Z}{f_x}] [Y = \frac{(v - c_y) \times Z}{f_y}]
- Kết quả thu được ((X, Y, Z)) chính là vị trí tọa độ 3D thực tế của khối trụ đối với camera (đơn vị cùng với (D_{real})).

BƯỚC 5: Mã nguồn Python mẫu (Chạy trực tiếp trên Pi 4B)

```
import cv2
import numpy as np

# 1. Các thông sô´đâù vào
# D_real: Đường kính thực tê´của khôí trụ (ví dụ: 0.1 mét = 10 cm)
D_real = 0.1

# Ma trận Intrinsics K (lấý từ bước Calibrate camera)
fx = 600.0  # Tiêu cự fx (pixel)
fy = 600.0  # Tiêu cự fy (pixel)
cx = 320.0  # Tâm quang học cx (với ảnh 640x480)
cy = 240.0  # Tâm quang học cy

# Khởi tạo camera
cap = cv2.VideoCapture(0)
```

```
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # 2. Xử lý ảnh tách nền bảng trắng
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    # Giả sử khối trụ có màu tối/màu sắc (khác màu trắng của bảng)
    # Lấy ngưỡng lọc loại bỏ màu trắng của bảng
    lower_bound = np.array()   # Điều chỉnh dải HSV theo màu khối trụ
    upper_bound = np.array()
    mask = cv2.inRange(hsv, lower_bound, upper_bound)

    # 3. Tìm contour khối trụ
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    if contours:
        # Chọn contour lớn nhất
        c = max(contours, key=cv2.contourArea)
        if cv2.contourArea(c) > 500: # Lọc nhiễu
            x, y, w, h = cv2.boundingRect(c)

            # Tọa độ tâm pixel 2D
            u = x + w / 2.0
            v = y + h / 2.0
            w_px = float(w)

            # 4. Tính toán tọa độ 3D (X, Y, Z)
            Z = (fx * D_real) / w_px
            X = ((u - cx) * Z) / fx
            Y = ((v - cy) * Z) / fy

            # Vẽ khung bao và hiển thị tọa độ
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.circle(frame, (int(u), int(v)), 5, (0, 0, 255), -1)
            text = f"X: {X:.2f}m, Y: {Y:.2f}m, Z: {Z:.2f}m"
            cv2.putText(frame, text, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.6, (0, 255, 0), 2)

    cv2.imshow("3D Position Estimation", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break

cap.release()
cv2.destroyAllWindows()
```

💡 Bạn có muốn mình hướng dẫn chi tiết cách thực hiện bước **Hiệu chỉnh Camera (Camera Calibration)** bằng bàn cờ caro trong OpenCV không?